

**Design & Implementation of An Interactive Visualisation System for Optimisation of
Hospital Resource Allocation.**

CIS4144 Data Analytics & Visualisation

Dr Huaizhong Zhang

Kate Forshaw

25259628

Word count: 2248

Contents

1. Introduction	3
1.1. Background	3
1.2. Aim & Objectives	4
2. System Design	5
3. Data Cleaning	7
3.1. Dataset Overview	7
3.2. Cleaning Process	7
4. Interactive Visualisation – EDA	9
5. Optimisation for Resource Allocation	19
5.1. Data Preparation	19
5.2. ML Method	19
5.3. Model Evaluation	19
6. Discussion	23
6.1. Main Findings & Recommendations	23
6.2. Limitations & Improvements	23
7. Conclusion	24
8. References	25
9. Appendices	30

Design & Implementation of An Interactive Visualisation System for Optimisation of Hospital Resource Allocation.

1.1. Background

Hospitals face persistent operational pressures due to fluctuating patterns in patient admissions. Demand can rise unpredictably, creating sudden surges that strain capacity and force hospitals to accelerate discharges to accommodate incoming patients (Reid et al., 1999; Sharma et al., 2008). Seasonal peaks – particularly during winter – further intensify this pressure, contributing to overcrowding, delays in care, and increased demand for clinical resources (Morbey et al., 2020). Emergency admissions require immediate intervention, whereas routine or urgent cases can be deferred, creating additional complexity in prioritising patient flow (NHS Midlands & Lancashire, 2026). When resources are not allocated effectively during these fluctuations, hospitals experience overworked staff, weakened quality of care, higher operational costs, and increased risks to patient safety (Malematja et al., 2025).

Data analytics plays a critical role in addressing these operational pressures by enabling hospitals to understand patterns in patient flow, anticipate periods of high demand, and allocate resources more effectively. By analysing historical admissions alongside demographic and clinical factors, hospitals can identify the drivers of resource utilisation and gain insight into how demand evolves over time. Interactive visualisations support administrators by transforming complex datasets into interpretable insights, comparing patient groups and uncovering relationships that may not be visible through static analysis alone (Weissgerber et al., 2016). Further, adopting predictive modelling will help them plan staffing levels, bed capacity, and equipment availability with greater accuracy (Balogun et al., 2022). In this context, developing an interactive analytical workflow provides a robust foundation for improving operational efficiency and supporting evidence-based resource planning.

1.2. Aim & Objectives

Aim: Design, implement, and evaluate an interactive analytical workflow that enables hospital administrators to explore patient admissions, clinical patterns, and resource-related trends through dynamic visualisations, and to evaluate predictive models that support evidence-based resource allocation.

Objectives:

- To clean, standardise, and prepare the historical admissions dataset, resolving missing values, inconsistencies, and outliers to ensure reliable analysis and modelling.
- To conduct exploratory and interactive visual analysis of using dynamic plots that allow filtering, comparison, and detailed inspection of admissions, diagnoses, outcomes, and clinical indicators.
- To design a conceptual interactive visualisation system roadmap, outlining how interactive analytical components support historical decision making.
- To develop and evaluate machine learning models for predicting resource-related outcomes, supported by interactive visualisations.
- To assess the workflow's effectiveness in supporting operational decision making and provide recommendations for improving hospital resource allocation.

2. System Design

The system is designed as an interactive analytical workflow that enables hospital administrators to explore historical admissions, clinical characteristics, and resource-related patterns. It consists of four core components: data cleaning and preparation, interactive exploratory visualisation, predictive modelling, and evaluation. Each component contributes to a coherent pipeline that transforms raw hospital data into actionable insights (see figure 2.1.).

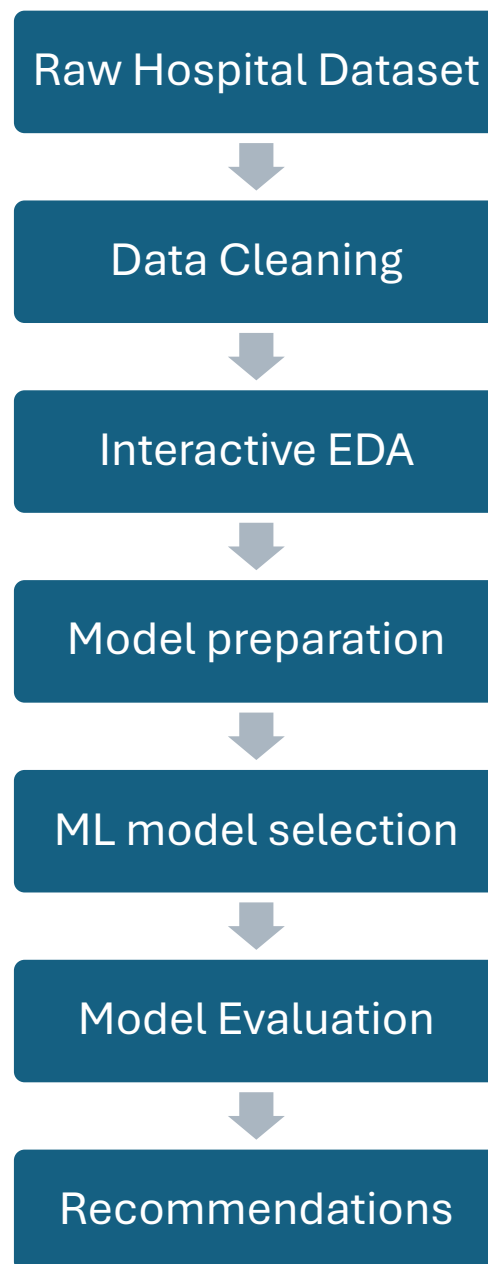


Figure 2.1. Block Diagram of Solution Roadmap

The data cleaning and preparation module standardises inconsistent formats, resolves missing values, and derives key variables such as duration of stay. This ensures analytical reliability and provides a stable foundation for both visualisation and modelling (Chu et al., 2016). The interactive visualisation module enables users to filter, compare, and explore trends dynamically through time-series plots, demographic distributions, clinical indicators, and correlation structures. This interactivity enhances interpretability and supports scenario-based reasoning (Packham et al, 2005).

The machine-learning module applies supervised models to predict resource-related outcomes, using engineered features derived from admissions, demographics, and clinical markers. Finally, the evaluation module presents model performance through interactive visualisations, enabling users to assess predictive value and operational relevance (Zhou et al., 2021). This design is feasible within a notebook-based environment and provides a practical, scalable framework for data-driven resource planning.

3. Data Cleaning

3.1. Dataset Overview

The dataset contained two years of cardiology admissions, including demographic information, admission details, laboratory results, comorbidities, and clinical outcomes. It had 15,757 rows of data and 56 columns, consisting of demographics, admission details, laboratory values, diagnoses and complications, and outcomes. Initial inspection revealed several structural issues: inconsistent column naming, mixed data types, missing admission and discharge dates, categorical fields encoded as single letters, and laboratory values stored as objects (see appendices 1, 2, 5 & 6). These inconsistencies required systematic cleaning to ensure the reliability of subsequent visualisation and modelling.

3.2. Cleaning Process

Column names were standardised to a consistent lowercase, underscore-separated format to support reproducible processing (see appendices 3 & 4). Admission and discharge dates were converted to UK datetime objects, with invalid entries coerced to missing values (see appendix 7). Categorical variables such as gender, locality, and admission type were recorded into descriptive labels, while laboratory measurements were converted to numeric types (see appendices 8, 9, 10 & 17).

A missingness profile showed substantial gaps in admission and discharge dates and moderate missingness in several laboratory fields (see appendices 11 & 12). Rows missing both dates were removed, while single date missingness was retained with indicator variables to preserve temporal information (see appendices 13 & 14). Laboratory values with moderate missingness were imputed using the median, and BNP – due to its high missingness – was retained with a missingness flag rather than imputed (see appendices 15 & 16).

A computed duration of stay variable was generated and validated against recorded values, revealing a consistent one-day difference reflecting standard hospital practice of

counting both admission and discharge days (see appendices 18 & 19). Duplicate admissions were removed to ensure each row represented a unique encounter (see appendix 20).

Further, outliers were detected across all numeric variables using an IQR function (see appendices 22 & 23). Several clinical markers, including creatinine, urea, glucose and BNP, displayed a high number of extreme values. These reflect genuine severe presentations typical of an advanced cardiology unit rather than data errors, so no outlier removal was performed (Rangaswami et al., 2019). Binary diagnosis flags also appeared as outliers due to the limitations of the IQR method for categorical 0/1 variables. Retaining these values preserves clinical validity of the dataset and ensures that subsequent visualisations and predictive models capture the full spectrum of patient severity.

The final cleaned dataset consisted of 11,738 rows and 61 columns and display settings were defined for plots, providing a consistent and analytically robust foundation for interactive visualisation and modelling (see appendix 21, 24 & 25).

4. Interactive Visualisation – EDA

The analysis begins with an exploration of admissions over time (see appendices 26-28). The monthly trend plot shows a steady rise in admissions across 2017 and 2018, with clear seasonal fluctuations and a pronounced increase approaching early 2019 (see figure 4.1.). When admissions are separated by type, emergency activity consistently exceeds OPD, yet both follow the same underlying trajectory, indicating shared temporal drivers of demand (see figure 4.2.). The rolling average smooths short-term variability and reinforces the presence of a genuine upward trend rather than random month-to-month noise (see figure 4.3.). Together, these three visuals provide a coherent picture of increasing service pressure over the study period and highlight the persistent dominance of emergency care in shaping overall hospital demand (Samadbeik et al., 2024).

Monthly Admissions Trend

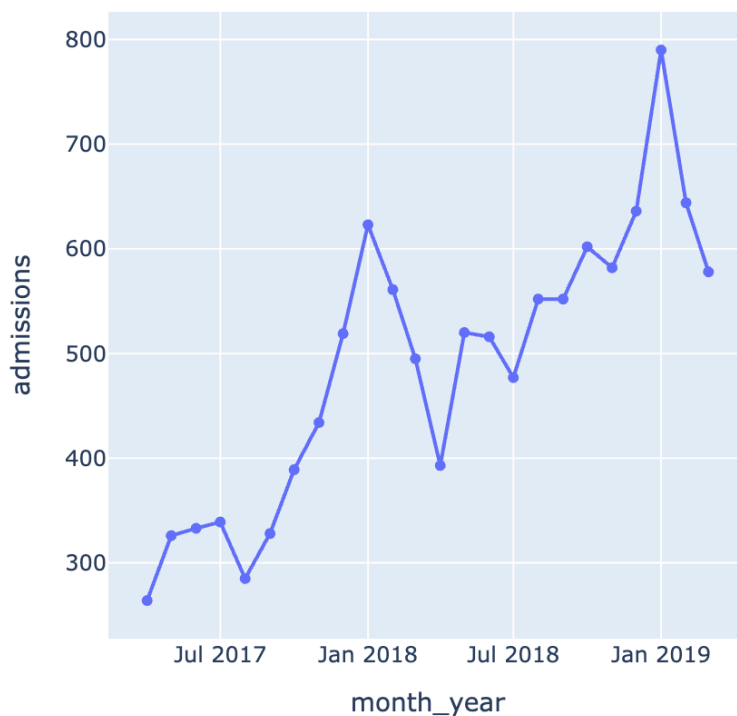


Figure 4.1. Interactive line chart of monthly admissions trend

Emergency vs OPD Admissions Over Time

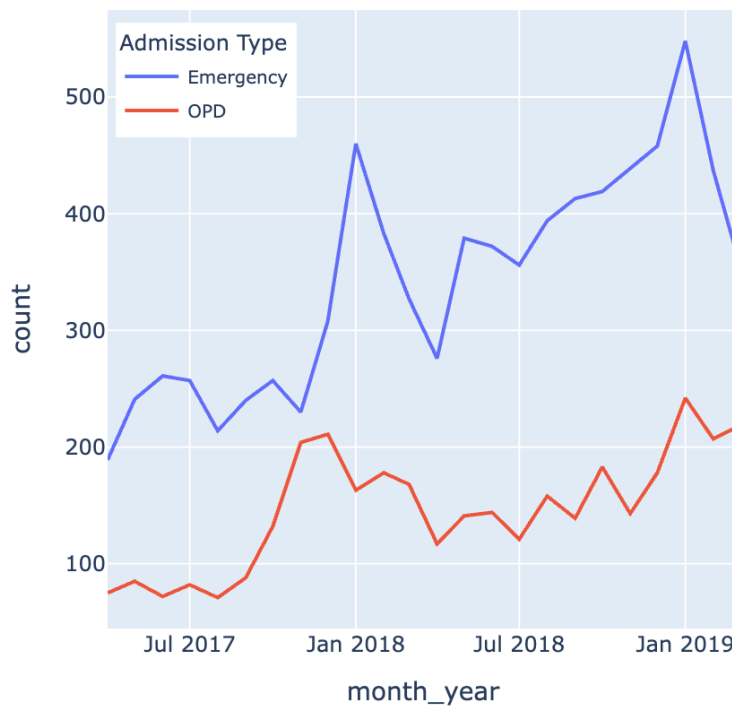


Figure 4.2. Interactive line graph of emergency vs OPD admissions over time

3-Month Rolling Average of Admissions



Figure 4.3. Interactive line chart of admissions 3-month rolling average

Turning to patient demographics, the age distribution shows a predominantly older population, with admissions clustering most densely between approximately 55 and 65 years (see appendix 29 & figure 4.4.). This concentration of middle-to-late adulthood admissions provides important clinical context for the subsequent analyses, as many of the biomarkers and conditions examined – including renal dysfunction, heart failure, and acute coronary syndromes – are strongly age-associated. The demographic profile therefore aligns with established evidence that cardiovascular and renal morbidity increase substantially with age (Singh et al., 2019; Dimopoulos et al., 2008).

Age Distribution of Patients

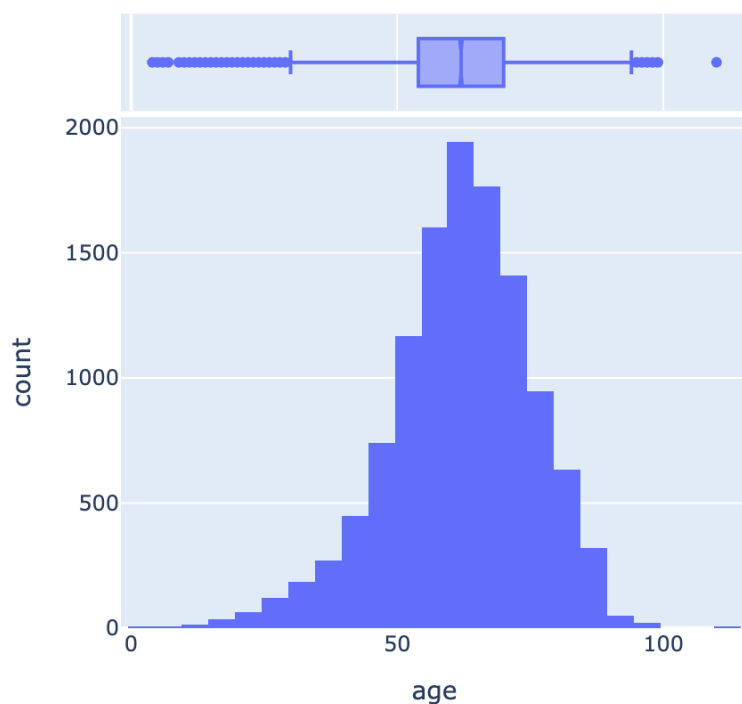


Figure 4.4. *Interactive histogram, boxplot, and KDE of patient age distributions*

The distribution of key biomarkers displays distinct variability patterns across the three panels (see appendix 30 & figure 4.5.). Creatinine, haemoglobin, and ejection fraction display moderate spread, with EF showing the widest range within this group. Glucose and urea exhibit more pronounced skewness, particularly glucose, which contains several high-value outliers consistent with acute metabolic disturbance (Corado, 2024). BNP, shown separately due to its

substantially larger scale, demonstrates extreme variability and numerous high outliers, reflecting the severity of cardiac dysfunction in a subset of patients (Romano, 2025). These patterns are reinforced in the pair plot (see appendix 31), where diffuse scatter and weak linear relationships between age, creatinine, EF, and duration of stay indicate that these variables interact in complex, non-linear ways rather than through simple pairwise associations (see figure 4.6.).

Distribution of Key Biomarkers

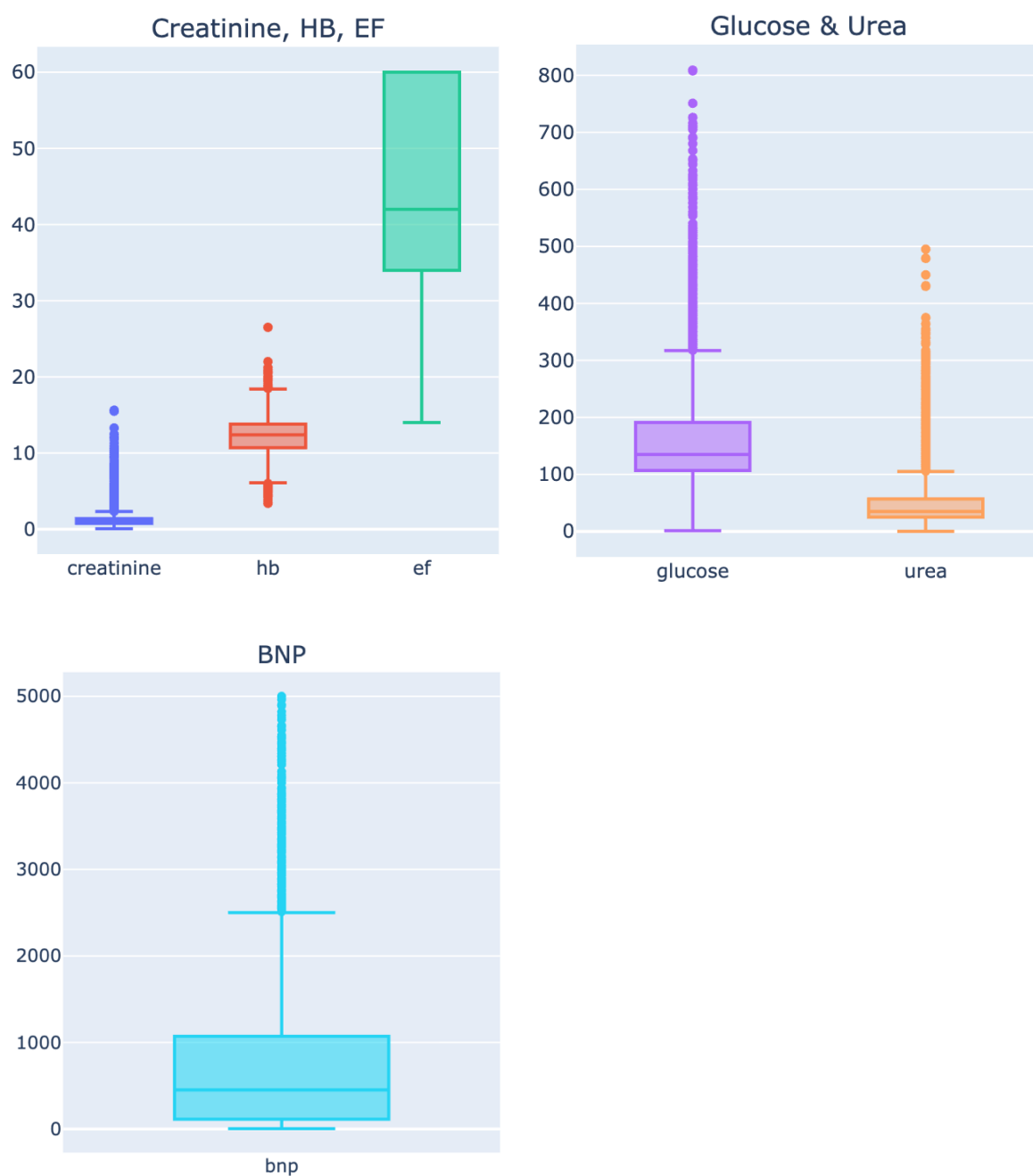


Figure 4.5. Interactive three-panel boxplots of biomarker distributions

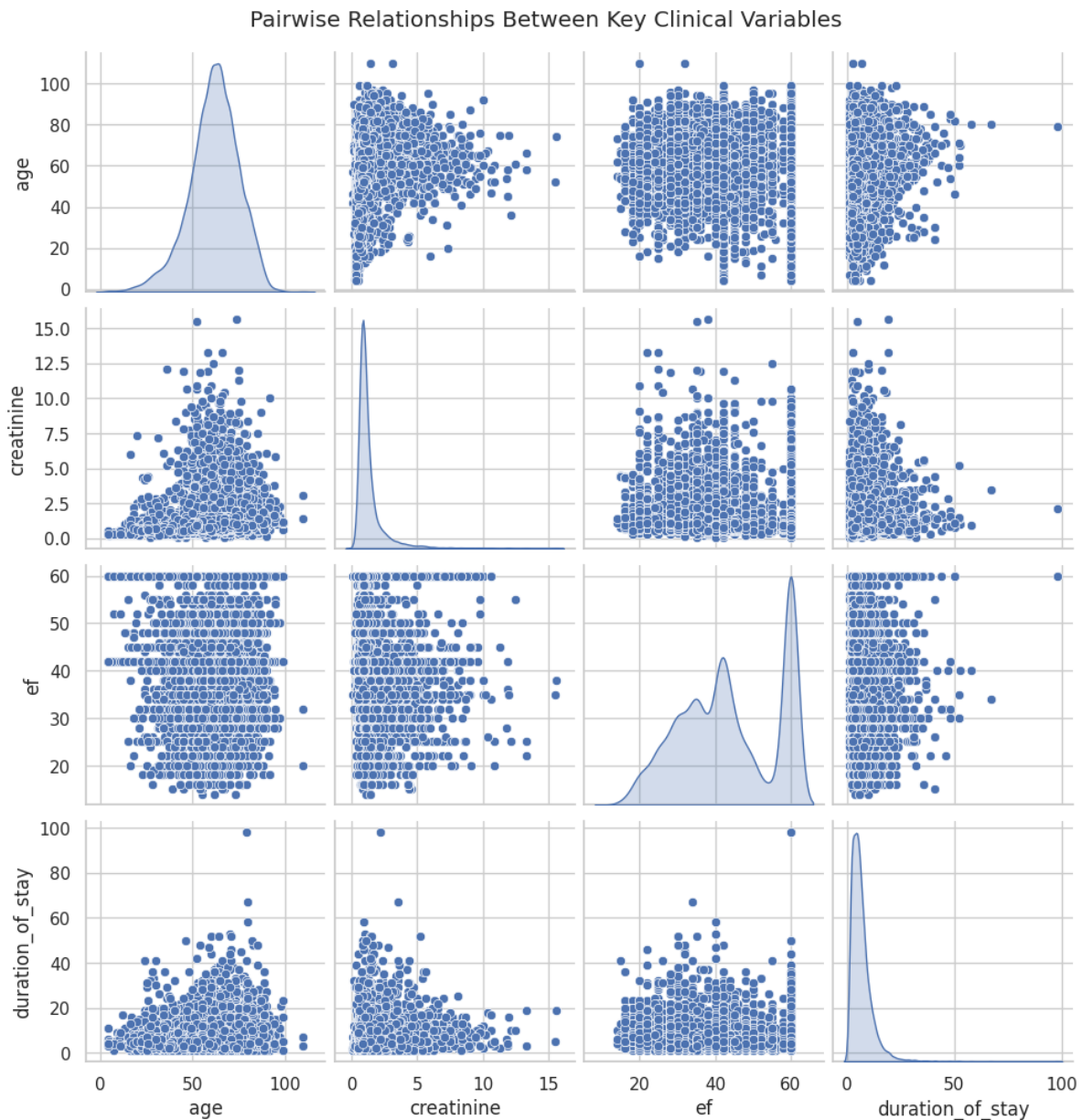


Figure 4.6. Pair plot of clinical variables

A deeper look at diagnostic patterns determines that ACS is the most common presentation, followed by heart failure, with AKI and STEMI forming a secondary but still clinically significant group (see appendix 32 & figure 4.7.). Valvular disease and stroke appear far less frequently, highlighting a clear concentration of admissions around acute coronary and decompensated cardiac conditions (Kotecha & Rakhit, 2016). The diagnosis co-occurrence heatmap reinforces this pattern (see appendix 33): the strongest association is observed

between ACS and STEMI, reflecting their shared pathophysiological pathway, while other diagnoses show only weak correlations (see figure 4.8.). This indicates that although comorbidities are present, they are not uniformly distributed across conditions, suggesting that most diagnoses occur as distinct clinical entities rather than tightly linked clusters (Awoyode et al., 2024).

Most Common Diagnoses

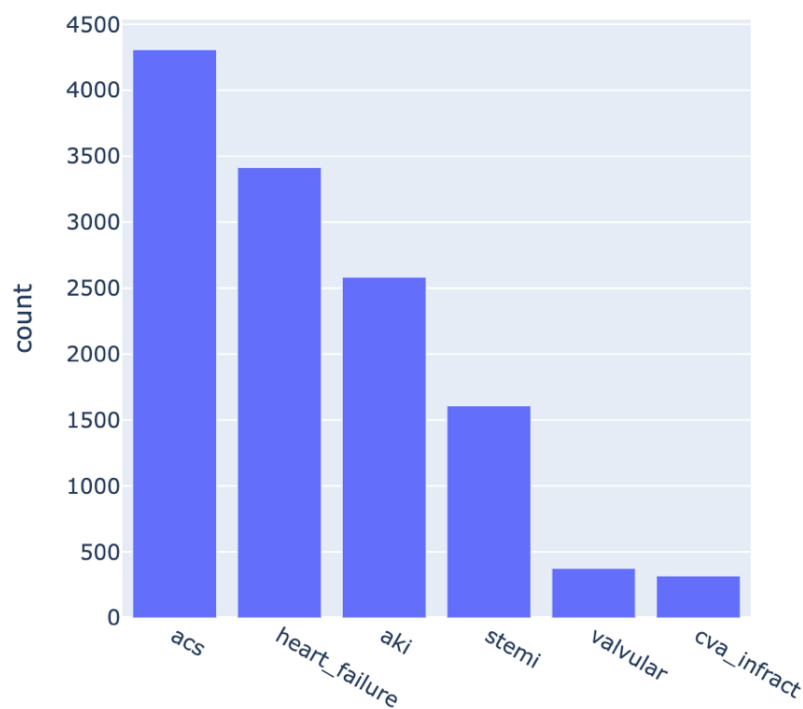


Figure 4.7. Interactive bar chart of most common diagnoses

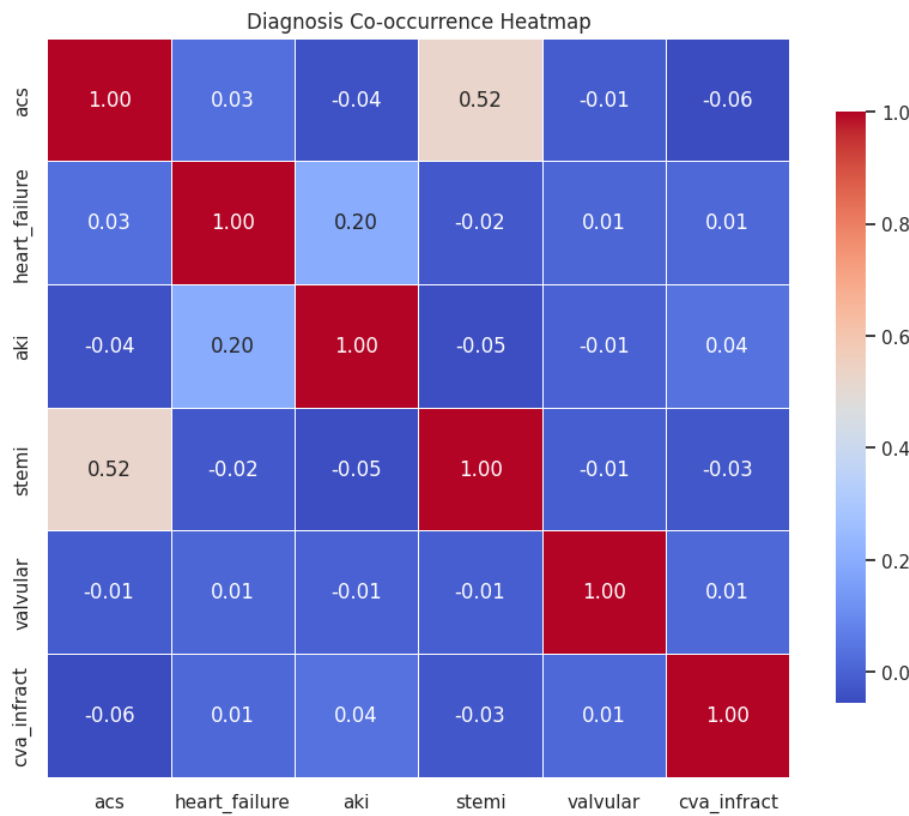


Figure 4.8. *Clustered heatmap of diagnosis co-occurrence*

The analysis of duration of stay by admission type highlights a clear divergence between emergency and OPD cases (see appendix 34). Emergency admissions exhibit substantially longer and more variable stays, with a wide distribution and a long upper tail that reflects the complexity and severity of acute presentations (see figure 4.9.). In contrast, OPD admissions are short and tightly clustered, indicating low-acuity cases that require minimal inpatient time. This pattern aligns with the earlier temporal findings and reinforces the interpretation that emergency activity is the primary driver of extended hospital resource use.

Duration of Stay by Admission Type

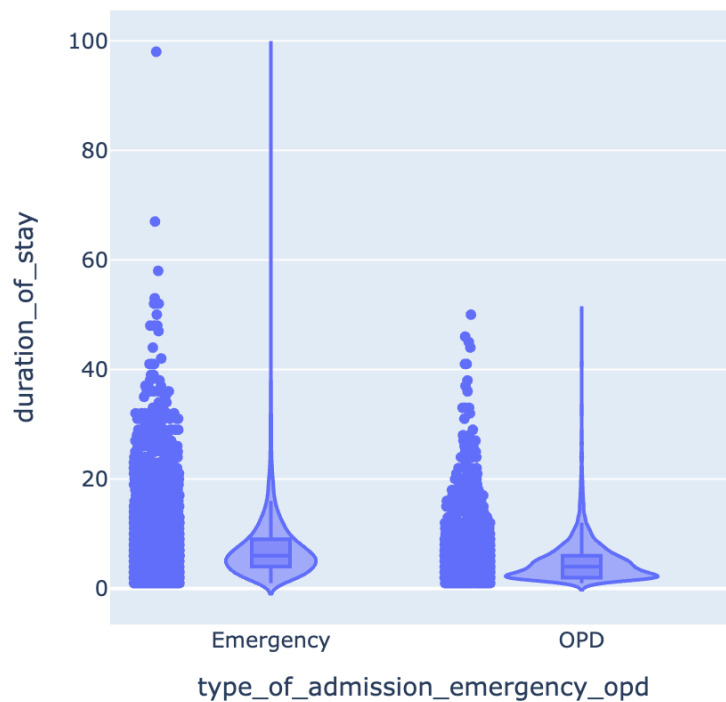


Figure 4.9. Interactive violin plot of duration of stay by admission type

Outcome patterns further contextualise these findings (see appendix 35). The distribution of patient outcomes shows that most admissions result in discharge, with DAMA and mortality representing only small proportions of cases (see figure 4.10.). This highly skewed pattern infers that, despite the clinical variability observed in biomarkers, diagnoses, and length of stay, most patients experience successful resolution of their admission (Wagner et al., 2013). The relatively low rates of DAMA and expiry are consistent with the predominance of acute but treatable cardiac presentations in this dataset.

Distribution of Patient Outcomes

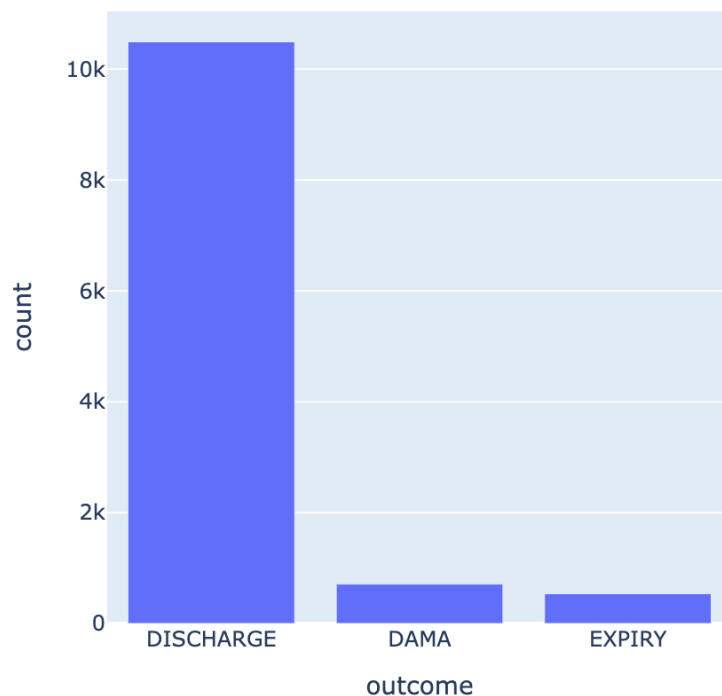


Figure 4.10. Interactive bar chart of patient outcomes distribution

Finally, the numeric clinical variable correlation heatmap and K-Means clustering provide a more structural view of the dataset (see appendices 36 & 37). The strongest relationship is observed between urea and creatinine, reflecting renal function as a coherent and internally consistent clinical dimension (Traynor et al., 2006), while the weak correlations with duration of stay indicate that length of stay (LOS) is shaped by multiple interacting factors rather than any single biomarker (Eskandari et al., 2022) (see figure 4.11.). The clustering analysis identifies three distinct patient groups based on creatinine and EF, suggesting the presence of meaningful clinical subtypes that differ in both renal impairment and cardiac function (see figure 4.12.). This segmentation supports the potential for stratified analysis and highlights underlying heterogeneity within the population.

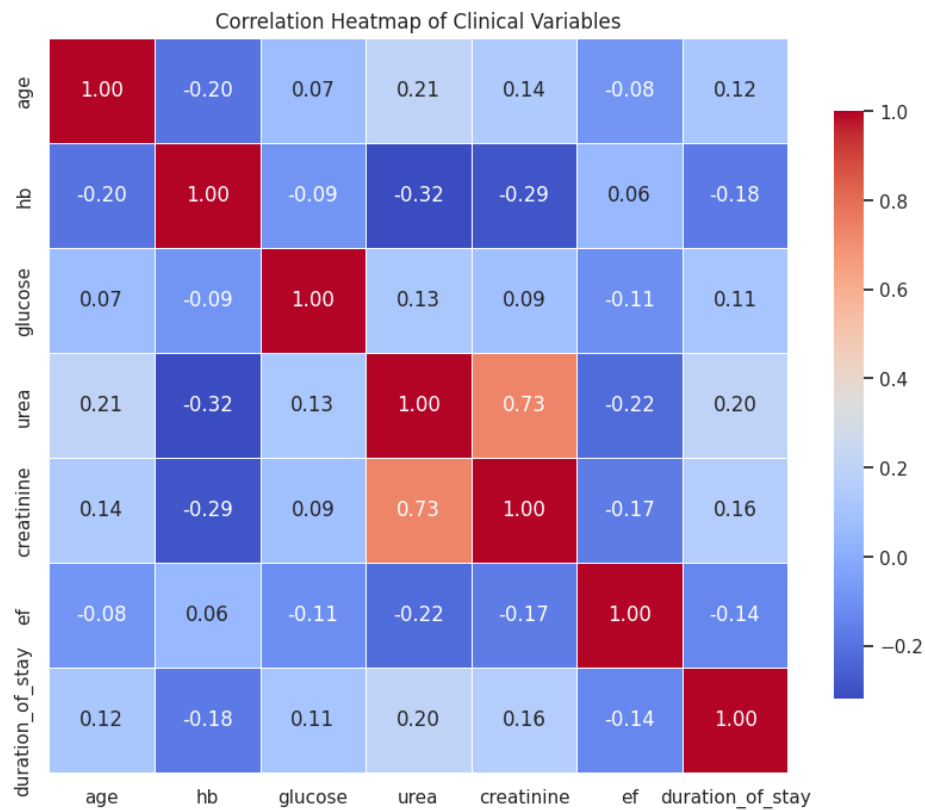


Figure 4.11. Correlation heatmap of clinical variables

K-Means Clusters of Clinical Profiles

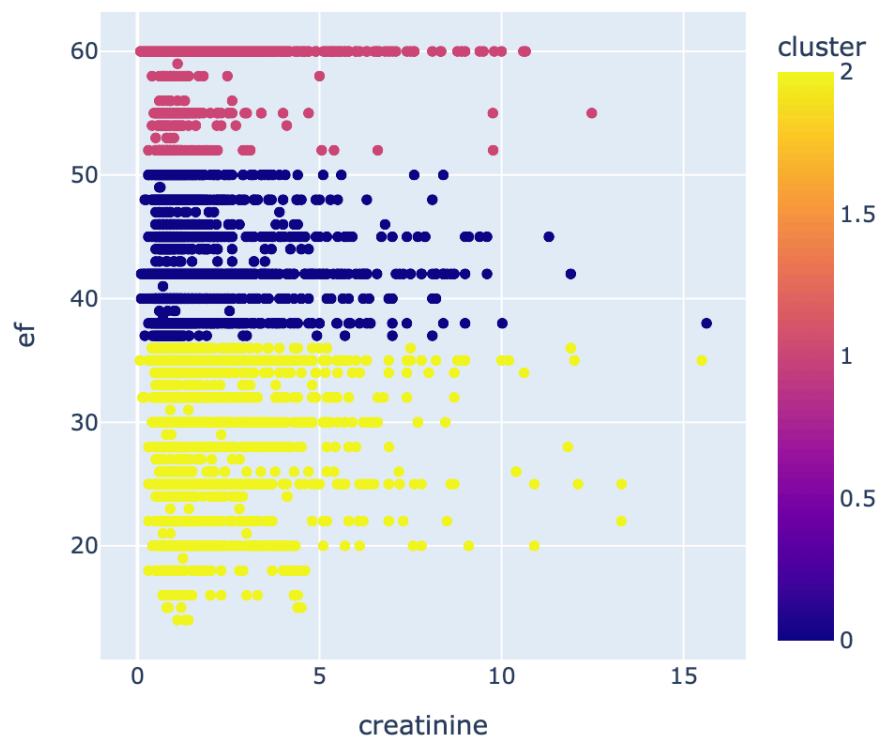


Figure 4.12. Interactive scatter plot of clinical profile K-means clusters

5. Optimisation for Resource Allocation

5.1. Data Preparation

Preparation began with importing required libraries for machine learning (see appendix 38). Duration of stay was selected as the target as LOS explicitly represents how long a patient occupies hospital resources, indicating it is an accurate and actionable measure for planning beds, staffing, and clinical workload (Karthikeyan et al., 2024) (see appendix 39). Leakage columns were identified and excluded from the feature variables as they represent information that would not be available at the time of prediction, leading to model failure in the real-world (Sasse et al., 2025). Numeric and categorical features were identified (see appendix 40 & 41).

Eighty percent of the data was used to train the models, while the remaining 20% was held to test model performance (see appendix 42). A fixed random state ensured that the split was reproducible. Scaling and one-hot encoding were implemented to account for any remaining missing values and convert the categorical labels into a numerical format, establishing proportional features and ordinal structure (Qiu et al., 2022) (see appendix 43).

5.2. ML Method

The ML models selected were Linear Regression (LR), Random Forest (RM), and Gradient Boosting (GB) (see appendix 44). A pipeline was built to fit each model: training them on the training data; predicting LOS on the test data; calculating MAE, RMSE, and R2; and storing the results in a comparison table (see appendix 45). MAE measures how many days the model is off (on average), RMSE determines how big the mistakes are, and R2 demonstrates how much of duration the model can explain (Khoshvaght et al., 2025).

5.3. Model Evaluation

All three models demonstrated similar performance, with GB achieving the lowest RMSE and highest R2, indicating slightly better predictive ability (see appendices 46 & 47). The MAE of approximately 2.9 days suggests that the models' predictions are within three

days of true LOS (see figure 5.3.1.). Despite the modest R2 values, LOS is difficult to predict as it is influenced by numerous unrecorded factors (Stone et al., 2022). Subsequent analysis was focussed on GB.

Model Performance Comparison (Regression Metrics)

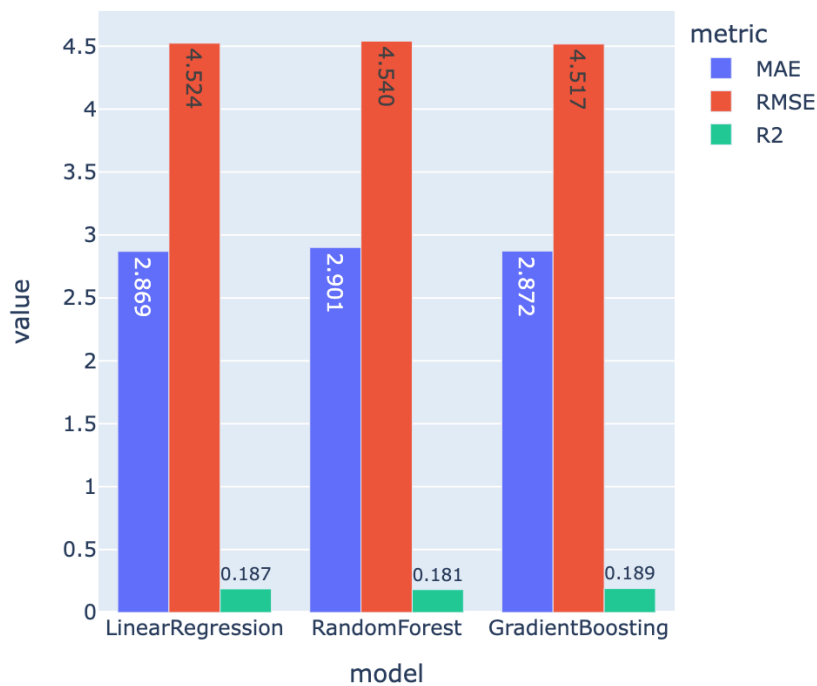


Figure 5.3.1. Interactive bar chart of model performance comparison

The true versus predicted plot presents that predictions broadly follow the diagonal trend, inferring GB captures the general relationship between admission-time features and LOS, however it tends to underestimate longer admissions where data is sparse (see appendix 48 & figure 5.3.2.). This is reinforced by the residual distribution, centring tightly around zero and displaying a right-skew, suggesting most predictions are reasonably accurate while significant underestimations occur rarely for prolonged stays (see appendix 49 & figure 5.3.3.). The feature importance plot demonstrates that GB relies primarily on clinical variables alongside admission type and age, highlighting that GB is learning plausible physiological and contextual signals rather than noise (see appendix 50 & figure 5.3.4.).

True vs Predicted Duration of Stay — GradientBoosting

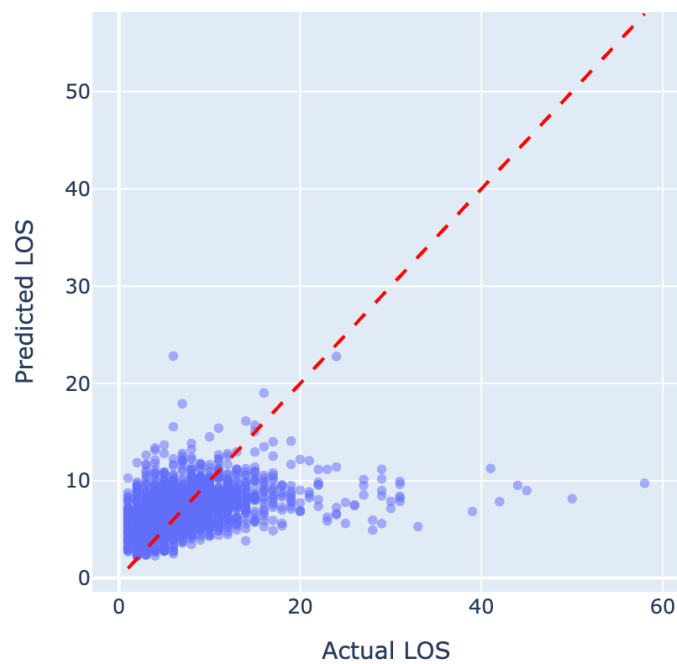


Figure 5.3.2. Interactive scatter plot of true vs predicted LOS

Residual Distribution — GradientBoosting

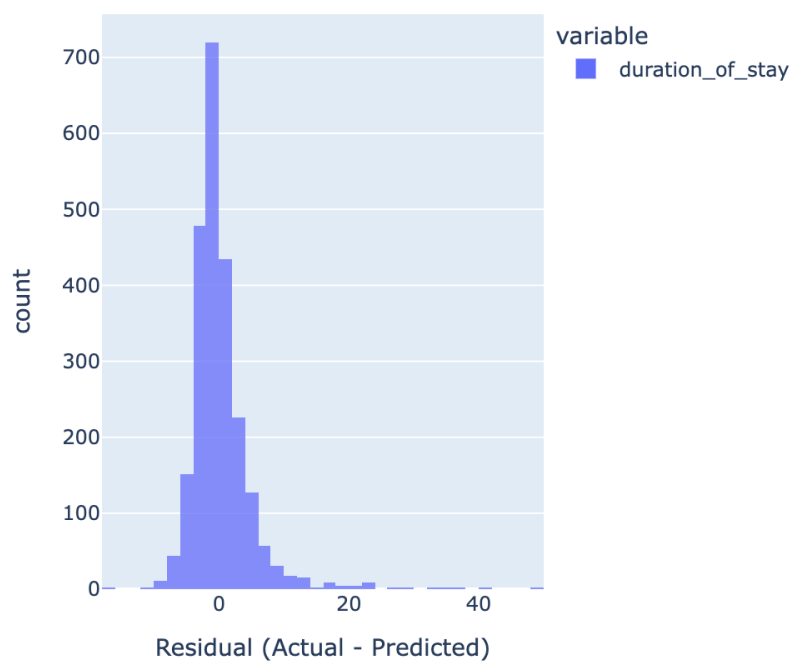


Figure 5.3.3. Interactive histogram of residual distribution

Top 20 Feature Importances — GradientBoosting

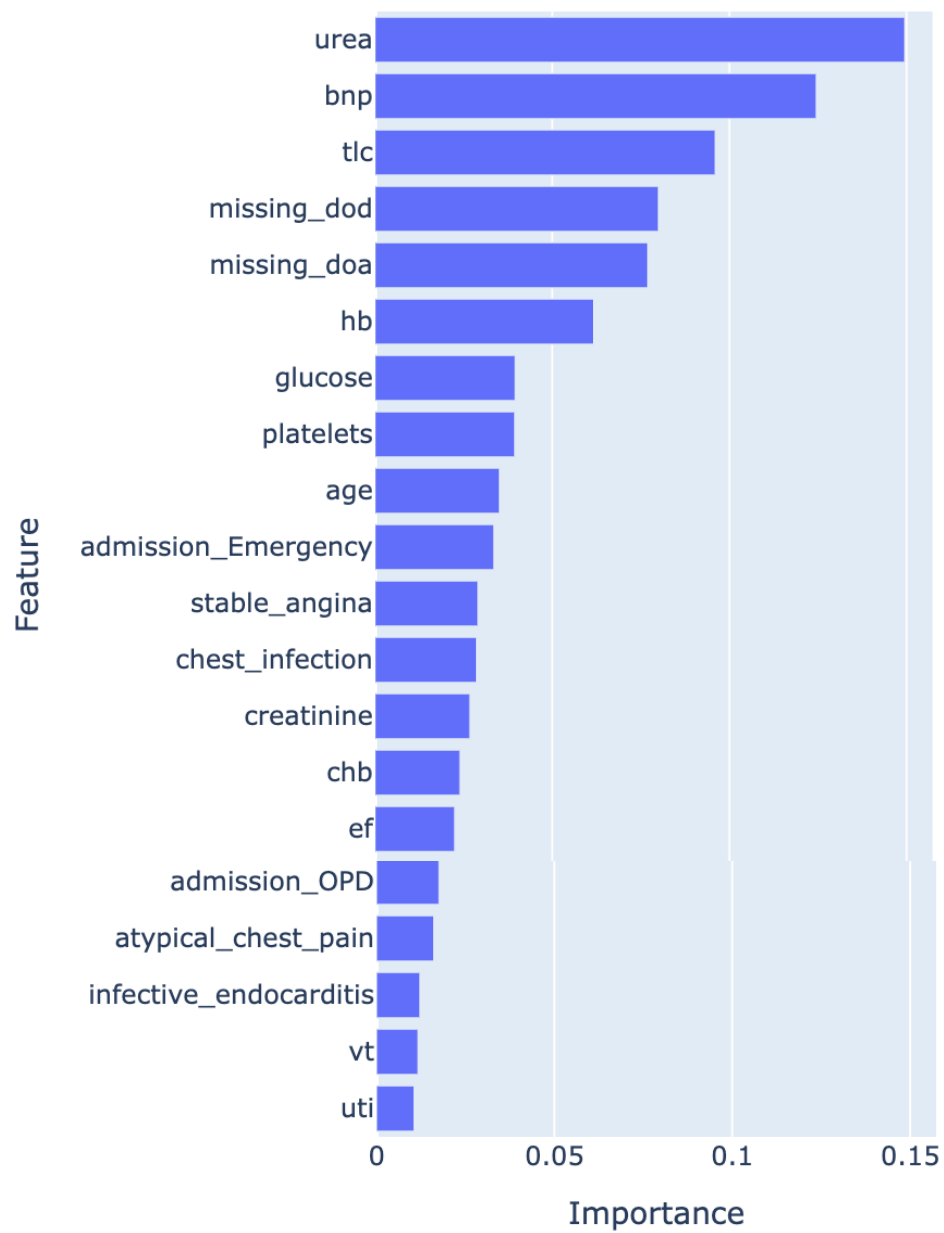


Figure 5.3.4. Interactive bar chart of feature importance

6. Discussion

6.1. Main Findings & Recommendations

The strong influence of biomarkers suggests that early identification of patients with abnormal values could enable proactive planning for longer admissions. Emergency activity represented the dominant driver of demand and is associated with substantially longer and more variable stays, illustrating a need for flexible surge capacity and targeted operational planning during peak periods. The predictive model provides clinically meaningful estimates of LOS within approximately three days, which could be incorporated into discharge planning and bed-management workflows to improve patient flow. Observed diagnostic patterns suggest that targeted care pathways for high-impact groups could streamline treatment processes and reduce avoidable delays. Moreover, historical patterns should be used to forecast expected surges in winter, adjusting staffing rosters, elective activity, and bed availability accordingly.

6.2. Limitations & Improvements

This interactive visualisation system has several limitations that should be acknowledged. The predictive performance of GB was modest, reflecting the inherent difficulty of modelling LOS using clinical variables alone. Missingness in key fields required imputation and indicators, which may have introduced bias. The dataset lacked operational variables such as staffing levels, bed occupancy, and weekend effects, all of which are known to influence LOS but were not available for modelling. The model systematically underestimated prolonged admissions, implying limited ability to capture rare, complex cases. Moreover, the dataset was drawn from a single cardiology centre, limiting generalisability, and the modelling approach did not incorporate temporal patterns or advanced feature engineering. Future work should focus on improving data completeness, integrating operational metrics, exploring more sophisticated models and engineered features, and expanding the dataset to enhance predictive accuracy and applicability.

7. Conclusion

Overall, the interactive visualisation system provides a robust and intuitive platform for exploring hospital admissions and clinical patterns. It enhances situational awareness, supports operational decision-making, and lays the groundwork for predictive modelling by revealing the key drivers of resource use. The system validates how interactive analytics can meaningfully improve understanding of patient flow and help hospitals plan resources more efficiently.

8. References

- AWOYODE, R., Ray, D.N., AKAMAH, J.A. and OKAFOR, H., 2024. Co-occurrence of Obstructive Coronary Artery Disease and Takotsubo Cardiomyopathy. *Cureus*, 16(4). Available from: https://assets.cureus.com/uploads/case_report/pdf/161020/20240527-21774-1n4t0d.pdf
- BALOGUN, E.D., OGUNSOLA, K.O. and OGUNMOKUN, A.S., 2022. Developing an advanced predictive model for financial planning and analysis using machine learning. *IRE journals*, 5(11), pp.320-328. Available from: <http://issn.org/resource/ISSN/2456-8880>
- CHU, X., ILYAS, I.F., KRISHNAN, S. and WANG, J., 2016, June. Data cleaning: Overview and emerging challenges. In *Proceedings of the 2016 international conference on management of data* (pp. 2201-2206). Available from: <https://doi.org/10.1145/2882903.2912574>
- CORADO, A.M., 2024. Hyperammonemia and inborn errors of metabolism. *Translational Pediatrics*, 13(2), pp.200-202. Available from: <https://doi.org/10.21037/tp-23-593>
- DIMOPOULOS, K., DILLER, G.P., KOLTSIDA, E., PIJUAN-DOMENECH, A., PAPADOPOULOU, S.A., BABU-NARAYAN, S.V., SALUKHE, T.V., PIEPOLI, M.F., POOLE-WILSON, P.A., BEST, N. and FRANCIS, D.P., 2008. Prevalence, predictors, and prognostic value of renal dysfunction in adults with congenital heart disease. *Circulation*, 117(18), pp.2320-2328. Available from: <https://doi.org/10.1161/CIRCULATIONAHA.107.734921>
- ESKANDARI, M., ALIZADEH BAHMANI, A.H., MARDANI-FARD, H.A., KARIMZADEH, I., OMIDFAR, N. and PEYMANI, P., 2022. Evaluation of factors that influenced the length of hospital stay using data mining techniques. *BMC*

Medical Informatics and Decision Making, 22(1), p.280. Available from:

[10.1186/s12911-022-02027-w](https://doi.org/10.1186/s12911-022-02027-w)

KARTHIKEYAN, P., BHARATH, M., VISHNUMRAM, M.K. and SRIRAM, B., 2024,

July. Forecasting patient length of stay for optimal hospital resource allocation.

In *2024 IEEE International Conference on Smart Power Control and Renewable*

Energy (ICSPCRE) (pp. 1-6). IEEE. Available from:

<https://doi.org/10.1109/ICSPCRE62303.2024.10675160>

KHOSHVAGHT, H., PERMALA, R.R., RAZMJOU, A. and KHIADANI, M., 2025. A

critical review on selecting performance evaluation metrics for supervised machine

learning models in wastewater quality prediction. *Journal of Environmental Chemical*

Engineering, 13(6), p.119675. Available from:

<https://doi.org/10.1016/j.jece.2025.119675>

KOTECHA, T. and RAKHIT, R.D., 2016. Acute coronary syndromes. *Clinical*

Medicine, 16(6), pp.s43-s48. Available from: [10.7861/clinmedicine.16-6s-s43](https://doi.org/10.7861/clinmedicine.16-6s-s43)

MALEMATJA, D.N., NKOSI, E.M. and NENE, S.E., 2025. The impact of insufficient

resources on the quality-of-service delivery at a primary healthcare clinic in

Limpopo. *curationis*, 48(1), p.2696. Available from: [10.4102/curationis.v48i1.2696](https://doi.org/10.4102/curationis.v48i1.2696)

MORBEY, R.A., CHARLETT, A., LAKE, I., MAPSTONE, J., PEBODY, R., SEDGWICK,

J., SMITH, G.E. and ELLIOT, A.J., 2020. Can syndromic surveillance help forecast

winter hospital bed pressures in England?. *PLoS One*, 15(2), p.e0228804. Available

from:

<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0228804#sec005>

NHS MIDLANDS & LANCASHIRE, 2026. Urgent and emergency healthcare: the

differences explained. *Staffordshire and Stoke-on-Trent, Integrated Care*

Board [online]. Available from: <https://staffsstone.icb.nhs.uk/improving-urgent-and-emergency-care/urgent-and-emergency-healthcare-the-differences-explained/>

PACKHAM, I.S., RAFIQ, M.Y., BORTHWICK, M.F. and DENHAM, S.L., 2005.

Interactive visualisation for decision support and evaluation of robustness—in theory and in practice. *Advanced Engineering Informatics*, 19(4), pp.263-280. Available from: <https://doi.org/10.1016/j.aei.2005.07.006>

QIU, S., LIU, Q., ZHOU, S. and HUANG, W., 2022. Adversarial attack and defense technologies in natural language processing: A survey. *Neurocomputing*, 492, pp.278-307. Available from: <https://doi.org/10.1016/j.neucom.2022.04.020>

RANGASWAMI, J., BHALLA, V., BLAIR, J.E., CHANG, T.I., COSTA, S., LENTINE, K.L., LERMA, E.V., MEZUE, K., MOLITCH, M., MULLENS, W. and RONCO, C., 2019. Cardiorenal syndrome: classification, pathophysiology, diagnosis, and treatment strategies: a scientific statement from the American Heart Association. *Circulation*, 139(16), pp.e840-e878. Available from: <https://doi.org/10.1161/CIR.0000000000000664>

ROMANO, M., 2025. Heart failure syndromes: different definitions of different diseases—do we need separate guidelines? A narrative review. *Journal of Clinical Medicine*, 14(14), p.5090. Available from: [10.3390/jcm14145090](https://doi.org/10.3390/jcm14145090)

REID, F.D., COOK, D.G. and MAJEED, A., 1999. Explaining variation in hospital admission rates between general practices: cross sectional study. *Bmj*, 319(7202), pp.98-103. Available from: https://www.bmj.com/content/319/7202/98.short?casa_token=OYpHZPS2bZUAAAAA:6_xj7tgbFcgAn5xBDDvI6OIY7YRtOZeOLxV068TjcGk9qoD_hy8kpxear2BvRRi_AtxJ-1RFLGxZ

- SAMADBEIK, M., STAIB, A., BOYLE, J., KHANNA, S., BOSLEY, E., BODNAR, D., LIND, J., AUSTIN, J.A., TANNER, S., MESHKAT, Y. and DE COURTEN, B., 2024. Patient flow in emergency departments: a comprehensive umbrella review of solutions and challenges across the health system. *BMC health services research*, 24(1), p.274. Available from: [10.1186/s12913-024-10725-6](https://doi.org/10.1186/s12913-024-10725-6)
- SASSE, L., NICOLAISEN-SOBESKY, E., DUKART, J., EICKHOFF, S.B., GOTZ, M., HAMDAN, S., KOMEYER, V., KULKARNI, A., LAHNAKOSKI, J.M., LOVE, B.C. and RAIMONDO, F., 2025. Overview of leakage scenarios in supervised machine learning. *Journal of Big Data*, 12(1), p.135. Available from: <https://link.springer.com/article/10.1186/s40537-025-01193-8>
- SHARMA, R., STANO, M. and GEHRING, R., 2008. Short-term fluctuations in hospital demand: implications for admission, discharge, and discriminatory behavior. *The Rand journal of economics*, 39(2), pp.586-606. Available from: <https://doi.org/10.1111/j.0741-6261.2008.00029.x>Digital Object Identifier (DOI)
- SINGH, G.M., BECQUART, N., CRUZ, M., ACEVEDO, A., MOZAFFARIAN, D. and NAUMOVA, E.N., 2019. Spatiotemporal and demographic trends and disparities in cardiovascular disease among older adults in the United States based on 181 million hospitalization records. *Journal of the American Heart Association*, 8(21), p.e012727. Available from: <https://doi.org/10.1161/JAHA.119.012727>
- STONE, K., ZWIGGELAAR, R., JONES, P. and MAC PARTHALAIN, N., 2022. A systematic review of the prediction of hospital length of stay: Towards a unified framework. *PLOS digital health*, 1(4), p.e0000017. Available from: [10.1371/journal.pdig.0000017](https://doi.org/10.1371/journal.pdig.0000017)

TRAYNOR, J., MACTIER, R., GEDDES, C.C. and FOX, J.G., 2006. How to measure renal function in clinical practice. *Bmj*, 333(7571), pp.733-737. Available from:

[10.1136/bmj.38975.390370.7C](https://doi.org/10.1136/bmj.38975.390370.7C)

WAGNER, J., GABLER, N.B., RATCLIFFE, S.J., BROWN, S.E., STROM, B.L. and HALPERN, S.D., 2013. Outcomes among patients discharged from busy intensive care units. *Annals of internal medicine*, 159(7), pp.447-455. Available from:

[10.7326/0003-4819-159-7-201310010-00004](https://doi.org/10.7326/0003-4819-159-7-201310010-00004)

WEISSGERBER, T.L., GAROVIC, V.D., SAVIC, M., WINHAM, S.J. and MILIC, N.M., 2016. From static to interactive: transforming data visualization to improve transparency. *PLoS biology*, 14(6), p.e1002484. Available from:

<https://doi.org/10.1371/journal.pbio.1002484>

ZHOU, J., GANDOMI, A.H., CHEN, F. and HOLZINGER, A., 2021. Evaluating the quality of machine learning explanations: A survey on methods and metrics. *Electronics*, 10(5), p.593. Available from:

<https://doi.org/10.3390/electronics10050593>

9. Appendices

Appendix 1 Import, load and summarise dataset code

```
#import pandas library
import pandas as pd

#load dataset
df3 = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/HDHI Admission data.csv")
df3.head()
```

Appendix 2 Summarise dataset output

	SNO	MRD No.	D.O.A	D.O.D	AGE	GENDER	RURAL	TYPE OF ADMISSION- EMERGENCY/OPD	month year	DURATION OF STAY	..
0	1	234735	4/1/2017	4/3/2017	81	M	R	E	Apr-17	3	.
1	2	234696	4/1/2017	4/5/2017	65	M	R	E	Apr-17	5	.
2	3	234882	4/1/2017	4/3/2017	53	M	U	E	Apr-17	3	.
3	4	234635	4/1/2017	4/8/2017	67	F	U	E	Apr-17	8	.
4	5	234486	4/1/2017	4/23/2017	60	F	U	E	Apr-17	23	.

CONGENITAL	UTI	NEURO CARDIOGENIC SYNCOPE	ORTHOSTATIC	INFECTIVE ENDOCARDITIS	DVT	CARDIOGENIC SHOCK	SHOCK
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Appendix 3 Standardise column names code

```
#standardise column names
df3.columns = (
    df3.columns
    .str.strip()
    .str.lower()
    .str.replace(r'[^a-zA-Z0-9]+', '_', regex=True)
    .str.replace('_', '-', regex=True)
    .str.rstrip('-')
)

df3.columns
```

Appendix 4 Column names output

```
Index(['sno', 'mrd_no', 'd_o_a', 'd_o_d', 'age', 'gender', 'rural',
      'type_of_admission_emergency_opd', 'month_year', 'duration_of_stay',
      'duration_of_intensive_unit_stay', 'outcome', 'smoking', 'alcohol',
      'dm', 'htn', 'cad', 'prior_cmp', 'ckd', 'hb', 'tlc', 'platelets',
      'glucose', 'urea', 'creatinine', 'bnp', 'raised_cardiac_enzymes', 'ef',
      'severe_anaemia', 'anaemia', 'stable_angina', 'acs', 'stemi',
      'atypical_chest_pain', 'heart_failure', 'hfref', 'hfnef', 'valvular',
      'chb', 'sss', 'aki', 'cva_infract', 'cva_bleed', 'af', 'vt', 'psvt',
      'congenital', 'uti', 'neuro_cardiogenic_syncope', 'orthostatic',
      'infective_endocarditis', 'dvt', 'cardiogenic_shock', 'shock',
      'pulmonary_embolism', 'chest_infection'],
      dtype='object')
```

Appendix 5 Information and sample of dataset code

```
#import numpy library
import numpy as np

#initial inspection
df3.info()
df3.sample(5)
```



```

23  urea                    15516 non-null object
24  creatinine              15510 non-null object
25  bnp                     7316 non-null object
26  raised_cardiac_enzymes  15757 non-null int64
27  ef                      14252 non-null object
28  severe_anaemia          15757 non-null int64
29  anaemia                 15757 non-null int64
30  stable_angina           15757 non-null int64
31  acs                     15757 non-null int64
32  stemi                   15757 non-null int64
33  atypical_chest_pain     15757 non-null int64
34  heart_failure           15757 non-null int64
35  hfref                   15757 non-null int64
36  hfnef                   15757 non-null int64
37  valvular                15757 non-null int64
38  chb                     15757 non-null int64
39  sss                     15757 non-null int64
40  aki                     15757 non-null int64
41  cva_infract             15757 non-null int64
42  cva_bleed               15757 non-null int64
43  af                      15757 non-null int64
44  vt                      15757 non-null int64
45  psvt                    15757 non-null int64
46  congenital              15757 non-null int64
47  uti                     15757 non-null int64
48  neuro_cardiogenic_syncope 15757 non-null int64
49  orthostatic             15757 non-null int64
50  infective_endocarditis  15757 non-null int64

51  dvt                    15757 non-null int64
52  cardiogenic_shock       15757 non-null int64
53  shock                   15757 non-null int64
54  pulmonary_embolism      15757 non-null int64
55  chest_infection          15757 non-null object
dtypes: int64(39), object(17)
memory usage: 6.7+ MB

```

	sno	mrd_no	d_o_a	d_o_d	age	gender	rural	type_of_admission_emergency_opd
14408	14409	666931	07/02/2019	11/02/2019	45	M	R	E
11240	11241	144426	04/10/2018	07/10/2018	41	M	U	E
5413	5414	410022	12/25/2017	12/29/2017	52	M	U	O
12973	12974	638367	15/12/2018	21/12/2018	55	M	U	E
8857	8858	8887	07/06/2018	12/06/2018	78	F	U	O

5 rows × 9 columns

type_of_admission_emergency_opd	month_year	duration_of_stay	...	congenital	uti	neuro_cardiogenic_syncope
E	Feb-19	5	...	0	0	0
E	Oct-18	4	...	0	0	0
O	Dec-17	5	...	0	0	0
E	Dec-18	7	...	0	0	0
O	Jun-18	6	...	0	0	0

orthostatic	infective_endocarditis	dvt	cardiogenic_shock	shock	pulmonary_embolism	chest_infection
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Appendix 7 Convert date columns code

```
#convert date columns to UK format
df3['d_o_a'] = pd.to_datetime(df3['d_o_a'], dayfirst=True, errors='coerce')
df3['d_o_d'] = pd.to_datetime(df3['d_o_d'], dayfirst=True, errors='coerce')
df3['month_year'] = pd.to_datetime(df3['month_year'], format='%b-%y', errors='coerce')
```

Appendix 8 Convert categorical fields code

```
#convert categorical fields
df3['gender'] = df3['gender'].astype('category')
df3['rural'] = df3['rural'].astype('category')
df3['type_of_admission_emergency_opd'] = df3['type_of_admission_emergency_opd'].astype('category')
df3['outcome'] = df3['outcome'].astype('category')

# MRD number should remain string (ID field)
df3['mrd_no'] = df3['mrd_no'].astype(str)
```

Appendix 9 Convert lab values code

```
#convert lab values to numeric
lab_cols = [
    'hb', 'tlc', 'platelets', 'glucose', 'urea', 'creatinine', 'bnp', 'ef'
]

for col in lab_cols:
    df3[col] = pd.to_numeric(df3[col], errors='coerce')
```

Appendix 10 Fix chest infection code

```
#fix chest infection data & data type
df3['chest_infection'] = (
    df3['chest_infection']
    .astype(str)
    .str.strip()
    .replace({'Yes': 1, 'No': 0})
)

df3['chest_infection'] = pd.to_numeric(df3['chest_infection'], errors='coerce').fillna(0).astype(int)
```

Appendix 11 Check missing values code

```
#check for missing values
df3.isna().sum()
```

Appendix 12 Missing values output

sno	0	cad	0
mrd_no	0	prior_cmp	0
d_o_a	3767	ckd	0
d_o_d	3770	hb	256
age	0	tlc	290
gender	0	platelets	294
rural	0	glucose	945
type_of_admission_emergency_opd	0	urea	244
month_year	0	creatinine	251
duration_of_stay	0	bnp	9081
duration_of_intensive_unit_stay	0	raised_cardiac_enzymes	0
outcome	0	ef	1599
smoking	0	severe_anaemia	0
alcohol	0	anaemia	0
dm	0	stable_angina	0
htn	0	acs	0

stemi	0		
atypical_chest_pain	0		
heart_failure	0		
hfref	0		
hfnf	0		
valvular	0		
chb	0		
sss	0		
aki	0	neuro_cardiogenic_syncope	0
cva_infract	0	orthostatic	0
cva_bleed	0	infective_endocarditis	0
af	0	dvt	0
vt	0	cardiogenic_shock	0
psvt	0	shock	0
congenital	0	pulmonary_embolism	0
uti	0	chest_infection	0

Appendix 13 Drop rows with both dates missing code

```
#drop rows missing both admission & discharge dates
df3 = df3.dropna(subset=['d_o_a', 'd_o_d'], how='all')
```

Appendix 14 Add indicators for singular missing dates code

```
#add missingness indicators for rows with one missing date
df3['missing_doa'] = df3['d_o_a'].isna().astype(int)
df3['missing_dod'] = df3['d_o_d'].isna().astype(int)
```

Appendix 15 Impute lab values with median code

```
#impute lab values with median
lab_cols = ['hb', 'tlc', 'platelets', 'glucose', 'urea', 'creatinine', 'ef']

for col in lab_cols:
    df3[col] = df3[col].fillna(df3[col].median())
```

Appendix 16 Add indicator for BNP

```
#add missingness indicator to BNP
#too high to drop or impute
df3['bnp_missing'] = df3['bnp'].isna().astype(int)
df3['bnp'] = pd.to_numeric(df3['bnp'], errors='coerce')
```

Appendix 17 Standardise categorical labels code

```
#standardise categorical labels for clarity
df3['gender'] = df3['gender'].map({'M': 'Male', 'F': 'Female'})
df3['rural'] = df3['rural'].map({'R': 'Rural', 'U': 'Urban'})
df3['type_of_admission_emergency_opd'] = df3['type_of_admission_emergency_opd'].map({
    'E': 'Emergency',
    'O': 'OPD'
})
```

Appendix 18 Validate duration and inspect mismatches code

```
#validate duration of stay
df3['computed_stay'] = (df3['d_o_d'] - df3['d_o_a']).dt.days
df3['stay_mismatch'] = df3['computed_stay'] != df3['duration_of_stay']

#inspect mismatches
df3[df3['stay_mismatch'] == True].head()
```

Appendix 19 Computed stay and mismatch output

duration_of_stay	computed_stay	stay_mismatch
3	59.0	True
5	120.0	True
3	59.0	True
8	212.0	True
23	NaN	True

Appendix 20 Remove duplicates code

```
#remove duplicates
df3 = df3.drop_duplicates(subset=['mrd_no', 'd_o_a'])
```

Appendix 21 Export cleaned dataset code

```
#export cleaned dataset
df3.to_csv("cleaned_hospital_data.csv", index=False)
print("Final shape:", df3.shape)
```

```
Final shape: (11738, 61)
```

Appendix 22 Outlier detection code

```
#detect outliers in numeric columns
numeric_cols = df3.select_dtypes(include=[np.number]).columns

#IQR function
def detect_outliers_iqr(data, cols):
    outlier_summary = {}

    for col in cols:
        Q1 = data[col].quantile(0.25)
        Q3 = data[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        outliers = data[(data[col] < lower_bound) | (data[col] > upper_bound)]

        outlier_summary[col] = {
            'lower_bound': lower_bound,
            'upper_bound': upper_bound,
            'num_outliers': outliers.shape[0]
        }

    return pd.DataFrame(outlier_summary).T
```

```
#run outlier detection
outlier_report = detect_outliers_iqr(df3, numeric_cols)
outlier_report
```

Appendix 23 Outlier detection output

	lower_bound	upper_bound	num_outliers
sno	-5724.750	22979.250	0.0
age	30.000	94.000	267.0
duration_of_stay	-4.500	15.500	584.0
duration_of_intensive_unit_stay	-5.000	11.000	547.0
smoking	0.000	0.000	585.0
alcohol	0.000	0.000	757.0
dm	-1.500	2.500	0.0
htn	-1.500	2.500	0.0
cad	-1.500	2.500	0.0
prior_cmp	0.000	0.000	1737.0
ckd	0.000	0.000	1165.0
hb	6.050	18.450	121.0
tlc	-0.050	21.150	590.0
platelets	-1.000	463.000	362.0
glucose	-19.000	317.000	630.0
urea	-23.000	105.000	1064.0
creatinine	-0.175	2.345	1226.0
bnp	-1325.000	2507.000	352.0
raised_cardiac_enzymes	0.000	0.000	2347.0
ef	-5.000	99.000	0.0
severe_anaemia	0.000	0.000	233.0
anaemia	0.000	0.000	2121.0
stable_angina	0.000	0.000	974.0
acs	-1.500	2.500	0.0
stemi	0.000	0.000	1607.0
atypical_chest_pain	0.000	0.000	305.0
heart_failure	-1.500	2.500	0.0
hfref	0.000	0.000	1734.0
hfnef	0.000	0.000	1690.0
valvular	0.000	0.000	375.0
chb	0.000	0.000	314.0
sss	0.000	0.000	75.0
aki	0.000	0.000	2584.0
cva_infract	0.000	0.000	318.0
cva_bleed	0.000	0.000	43.0

	lower_bound	upper_bound	num_outliers
af	0.000	0.000	540.0
vt	0.000	0.000	372.0
psvt	0.000	0.000	93.0
congenital	0.000	0.000	111.0
uti	0.000	0.000	714.0
neuro_cardiogenic_syncope	0.000	0.000	98.0
orthostatic	0.000	0.000	106.0
infective_endocarditis	0.000	0.000	23.0
dvt	0.000	0.000	168.0
cardiogenic_shock	0.000	0.000	537.0
shock	0.000	0.000	397.0
pulmonary_embolism	0.000	0.000	164.0
chest_infection	0.000	0.000	278.0
missing_doa	0.000	0.000	915.0
missing_dod	0.000	0.000	929.0
bnp_missing	-1.500	2.500	0.0
computed_stay	-7.500	20.500	1547.0

Appendix 24 Import EDA libraries code

```
#import libraries for EDA
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

Appendix 25 Set display settings for plots code

```
#display settings
#makes plots cleaner
sns.set(style='whitegrid')
plt.rcParams['figure.figsize'] = (10, 5)
```

Appendix 26 Monthly admissions trend plot code

```
#monthly admissions trend
#interactive line chart
#seasonal patterns, peaks, collapses, operational surges

monthly = df3.groupby('month_year').size().reset_index(name='admissions')

fig = px.line(monthly, x='month_year', y='admissions',
              title='Monthly Admissions Trend', markers=True)
fig.show()
```

Appendix 27 Emergency vs OPD admissions over time plot code

```
#emergency vs OPD admissions over time
#interactive line chart
#differences in demand streams, parallel trends, seasonal divergence
adm_type = df3.groupby(['month_year', 'type_of_admission_emergency_opd']).size().reset_index(name='count')

fig = px.line(adm_type, x='month_year', y='count',
              color='type_of_admission_emergency_opd',
              title='Emergency vs OPD Admissions Over Time')

fig.update_layout(
    legend_title_text='Admission Type',
    legend=dict(
        x=0.30, #move to left
        y=0.98, #move to top
        xanchor='right',
        yanchor='top',
        font=dict(size=10) #adjust legend text size
    )
)

fig.show()
```

Appendix 28 Rolling average trend plot code

```
#rolling average trend
#interactive line chart
#underlying trend, removes noise
monthly['rolling'] = monthly['admissions'].rolling(3).mean()

fig = px.line(monthly, x='month_year', y='rolling',
              title='3-Month Rolling Average of Admissions')

fig.show()
```

Appendix 29 Age distribution of patients plot code

```
#age distribution of patients
#histogram + box + KDE
#age skew, dominant age groups, clinical demographics
fig = px.histogram(df3, x='age', nbins=40, marginal='box',
                  title='Age Distribution of Patients')
fig.show()
```

Appendix 30 Biomarker distributions plot code

```
#biomarker distributions
#three-panel interactive boxplots
#outliers, variability, severity patterns
import plotly.subplots as sp
import plotly.graph_objects as go

#define biomarker groups
group1 = ['creatinine', 'hb', 'ef']
group2 = ['glucose', 'urea']
group3 = ['bnp']

#create 3-panel subplot layout
fig = sp.make_subplots(
    rows=1, cols=3,
    subplot_titles=("Creatinine, HB, EF", "Glucose & Urea", "BNP"),
    horizontal_spacing=0.08
)

#panel 1 – creatinine, hb, ef
for col in group1:
    fig.add_trace(go.Box(y=df3[col], name=col), row=1, col=1)

#panel 2 – glucose, urea
for col in group2:
    fig.add_trace(go.Box(y=df3[col], name=col), row=1, col=2)

#panel 3 – BNP only
fig.add_trace(go.Box(y=df3['bnp'], name='bnp'), row=1, col=3)

#layout adjustments
fig.update_layout(
    title="Distribution of Key Biomarkers",
    showlegend=False,
    height=500,
    width=1200
)

fig.show()
```

Appendix 31 Pair plot of clinical variables code

```
#pairplot of clinical variables
#advanced relationship exploration
#non-linear relationships, clusters, spread
sns.pairplot(
    df3[['age', 'creatinine', 'ef', 'duration_of_stay']],
    diag_kind='kde',
)
plt.suptitle('Pairwise Relationships Between Key Clinical Variables', y=1.02)
plt.show()
```

Appendix 32 Most common diagnoses plot code

```
#most common diagnoses
#interactive bar chart
#high-burden conditions, resource-intensive diagnoses
diagnosis_cols = ['acs', 'heart_failure', 'aki', 'stemi', 'valvular', 'cva_infract']

diag_counts = df3[diagnosis_cols].sum().reset_index()
diag_counts.columns = ['diagnosis', 'count']

fig = px.bar(diag_counts, x='diagnosis', y='count',
             title='Most Common Diagnoses')
fig.show()
```

Appendix 33 Diagnosis co-occurrence plot code

```
#diagnosis co-occurrence
#clustered heatmap
#comorbidity patterns, clinical overlap
plt.figure(figsize=(10,8))
sns.heatmap(
    df3[diagnosis_cols].corr(),
    cmap='coolwarm',
    annot=True,
    fmt=".2f",
    linewidths=0.5,
    cbar_kws={'shrink': 0.8}
)
plt.title('Diagnosis Co-occurrence Heatmap')
plt.show()
```

Appendix 34 Duration of stay by admission type plot code

```
#duration of stay by admission type
#interactive violin plot
#severity differences, variability, LOS patterns
fig = px.violin(
    df3,
    x='type_of_admission_emergency_opd',
    y='duration_of_stay',
    box=True,          #adds a mini boxplot inside
    points='all',      #shows individual points
    title='Duration of Stay by Admission Type'
)

fig.show()
```

Appendix 35 Patient outcomes distribution plot code

```
#patient outcomes distribution
#interactive bar chart
#mortality, DAMA, discharge patterns
outcome_counts = df3['outcome'].value_counts().reset_index()
outcome_counts.columns = ['outcome', 'count']

fig = px.bar(outcome_counts, x='outcome', y='count',
              title='Distribution of Patient Outcomes')
fig.show()
```

Appendix 36 Correlation heatmap of key numerical clinical variables code

```
#key numeric clinical variables
#correlation heatmap
#strong renal correlations, weak LOS correlations
num_cols = ['age', 'hb', 'glucose', 'urea', 'creatinine', 'ef', 'duration_of_stay']

plt.figure(figsize=(10,8))
sns.heatmap(
    df3[num_cols].corr(),
    cmap='coolwarm',
    annot=True,
    fmt=".2f",
    linewidths=0.5,
    cbar_kws={'shrink': 0.8}
)
plt.title('Correlation Heatmap of Clinical Variables')
plt.show()
```

Appendix 37 K-means clusters of clinical profiles plot code

```
#k-means clusters of clinical profiles
#interactive scatter plot
#patient subtypes, severity groups, clinical segmentation
from sklearn.cluster import KMeans

X = df3[['creatinine', 'ef']].dropna()
kmeans = KMeans(n_clusters=3, random_state=42).fit(X)
df3.loc[X.index, 'cluster'] = kmeans.labels_

fig = px.scatter(df3, x='creatinine', y='ef', color='cluster',
                 title='K-Means Clusters of Clinical Profiles')
fig.show()
```

Appendix 38 Import ML libraries code

```
#imports & setup for ML
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
```

Appendix 39 Define target and feature sets code

```
#define target and feature sets
target_col = 'duration_of_stay' #resource use via length of stay

leakage_cols = [
    'sno', 'mrd_no', 'd_o_a', 'd_o_d', 'month_year', 'outcome', 'computed_stay',
    'stay_mismatch', 'cluster', 'duration_of_intensive_unit_stay'
] #columns that should not be used as predictors

feature_cols = [c for c in df3.columns if c not in leakage_cols + [target_col]]

X = df3[feature_cols].copy()
y = df3[target_col].copy()
```

Appendix 40 Identify numeric and categorical features code

```
#identify numeric and categorical features
numeric_features = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_features = X.select_dtypes(include=['object', 'category']).columns.tolist()

print("Numeric features:", numeric_features)
print("Categorical features:", categorical_features)
```

Appendix 41 Numeric and categorical features output

```
Numeric features: ['age', 'smoking', 'alcohol', 'dm', 'htn', 'cad',
'prior_cmp', 'ckd', 'hb', 'tlc', 'platelets', 'glucose', 'urea',
'creatinine', 'bnp', 'raised_cardiac_enzymes', 'ef', 'severe_anaemia',
'anaemia', 'stable_angina', 'acs', 'stemi', 'atypical_chest_pain',
'heart_failure', 'hfref', 'hfnef', 'valvular', 'chb', 'sss', 'aki',
'cva_infract', 'cva_bleed', 'af', 'vt', 'psvt', 'congenital', 'uti',
'neuro_cardiogenic_syncope', 'orthostatic', 'infective_endocarditis',
'dvt', 'cardiogenic_shock', 'shock', 'pulmonary_embolism',
'chest_infection', 'missing_doa', 'missing_dod', 'bnp_missing']
Categorical features: ['gender', 'rural',
'type_of_admission_emergency_opd']
```

Appendix 42 Train-test split code

```
#train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

Appendix 43 Scaling and one-hot encoding code

```
#scaling + one-hot encoding
from sklearn.impute import SimpleImputer

numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)
```

Appendix 44 Define ML models code

```
#define multiple models to compare
models = {
    "LinearRegression": LinearRegression(),
    "RandomForest": RandomForestRegressor(
        n_estimators=200,
        random_state=42,
        n_jobs=-1
    ),
    "GradientBoosting": GradientBoostingRegressor(
        random_state=42
    )
}
```

Appendix 45 Train and evaluate models code

```
#train models & evaluate test data
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

results = []

for name, model in models.items():
    pipe = Pipeline(steps=[
        ('preprocess', preprocessor),
        ('model', model)
    ]) #fits each model

    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test) #predicts LOS

    mae = mean_absolute_error(y_test, y_pred)
    rmse = mean_squared_error(y_test, y_pred) ** 0.5
    r2 = r2_score(y_test, y_pred) #computes MAE, RMSE, r2

    results.append({
        'model': name,
        'MAE': mae,
        'RMSE': rmse,
        'R2': r2,
        'pipeline': pipe
    })

results_df = pd.DataFrame(results).drop(columns=['pipeline'])
results_df #stores in comparison table
```

Appendix 46 Test model results output

	model	MAE	RMSE	R2
0	LinearRegression	2.869449	4.523838	0.186935
1	RandomForest	2.900568	4.540000	0.181115
2	GradientBoosting	2.872158	4.517051	0.189373

Appendix 47 Model performance comparison plot code

```
#model performance comparison
#interactive bar chart
#visualise regression metrics
metrics_melted = results_df.melt(
    id_vars='model',
    value_vars=['MAE', 'RMSE', 'R2'],
    var_name='metric',
    value_name='value'
)

fig = px.bar(
    metrics_melted,
    x='model',
    y='value',
    color='metric',
    barmode='group',
    title='Model Performance Comparison (Regression Metrics)',
    text_auto='.3f'
)
fig.show()
```

Appendix 48 True vs predicted plot code

```
#true vs predicted
#interactive scatter
#how well the model fits across the LOS range
best_row = max(results, key=lambda x: x['R2'])
best_name = best_row['model']
best_pipe = best_row['pipeline']

y_pred_best = best_pipe.predict(X_test)

fig = px.scatter(
    x=y_test,
    y=y_pred_best,
    labels={'x': 'Actual LOS', 'y': 'Predicted LOS'},
    title=f'True vs Predicted Duration of Stay - {best_name}',
    opacity=0.5
)

fig.add_shape(
    type='line',
    x0=y_test.min(), y0=y_test.min(),
    x1=y_test.max(), y1=y_test.max(),
    line=dict(color='red', dash='dash')
)

fig.show()
```

Appendix 49 Residual distribution plot code

```
#residual distribution
#interactive histogram
#bias, skew, and model error structure
residuals = y_test - y_pred_best

fig = px.histogram(
    residuals,
    nbins=40,
    title=f'Residual Distribution - {best_name}',
    labels={'value': 'Residual (Actual - Predicted)'},
    opacity=0.75
)

fig.show()
```

Appendix 50 Feature importance plot code

```
#feature importance
#interactive bar chart
#only for tree-based models
import numpy as np
import pandas as pd
import plotly.express as px

#extract the trained model from the best pipeline
model = best_pipe.named_steps['model']

#proceed if the model supports feature importance
if hasattr(model, "feature_importances_"):

    #get feature names after preprocessing
    ohe = best_pipe.named_steps['preprocess'].named_transformers_['cat'].named_steps['onehot']
    cat_names = ohe.get_feature_names_out(categorical_features)

    #combine numeric + categorical names
    feature_names = np.concatenate([numeric_features, cat_names])

    #clean long one-hot encoded labels
    clean_feature_names = []
    for name in feature_names:
        if name.startswith("type_of_admission_emergency_opd_"):
            short = name.replace("type_of_admission_emergency_opd_", "admission_")

            clean_feature_names.append(name)

#build dataframe
importances = model.feature_importances_

fi_df = pd.DataFrame({
    'feature': clean_feature_names,
    'importance': importances
}).sort_values('importance', ascending=False)

#plot bar chart
fig = px.bar(
    fi_df.head(20),
    x='importance',
    y='feature',
    orientation='h',
    title=f"Top 20 Feature Importances - {best_name}",
    labels={'importance': 'Importance', 'feature': 'Feature'}
)

fig.update_layout(
    yaxis={'categoryorder': 'total ascending'},
    height=700
)

fig.show()
```

Appendix 51 Google Colab link if jupyter file does not open

https://colab.research.google.com/drive/1xili1zRqEg7gpZeYfdd_r2dszHnpftjO?usp=sharing